Undergraduate Lab Report of Jinan University

Course Title Experiment of Computer Organization Evaluation

Lab Name Performance Lab Instructor SUN Heng

Lab Address N116, Panyu Campus

Student Name, No.

李荣耀(2024103975)、陈抟飞(2024101329)、杨启承(2024101322)

College International School, Jinan University

Department Major Computer Science & Technology

Date 2025 / 12 / 17 Afternoon

1. Introduction

The assembly language in Lab 5 is one of the most effective means for gaining an understanding how the code will run in high performance. In this lab, students should optimize the performance of an application kernel function such as convolution or matrix transposition. It provides a clear demonstration of the properties of cache memories and gives them experience with low-level program optimization.

This is a team project (up to three students) which will be completed in two weeks.

2. Lab Instructions or Steps

The Image Processing Problem

This lab deals with optimizing memory intensive code. Image processing offers many examples of functions that can benefit from optimization. In this lab, we will consider two image processing operations: **rotate**, which rotates an image counter-clockwise by 90°, and **smooth**, which "smooths" or "blurs" an image.

For this lab, we will consider an image to be represented as a two-dimensional matrix M, where Mi,j denotes the value of (i, j)th pixel of M. Pixel values are triples of red, green, and blue (RGB) values. We will only consider square images. Let N denote the number of rows (or columns) of an image. Rows and columns are numbered, in C-style, from 0 to N-1.

Given this representation, the **rotate** operation can be implemented quite simply as the combination of the following two matrix operations:

• **Transpose**: For each (i, j) pair, Mi,j and Mj,i are interchanged.

• **Exchange rows**: Row i is exchanged with row N-1-i.

This combination is illustrated in Figure 1.

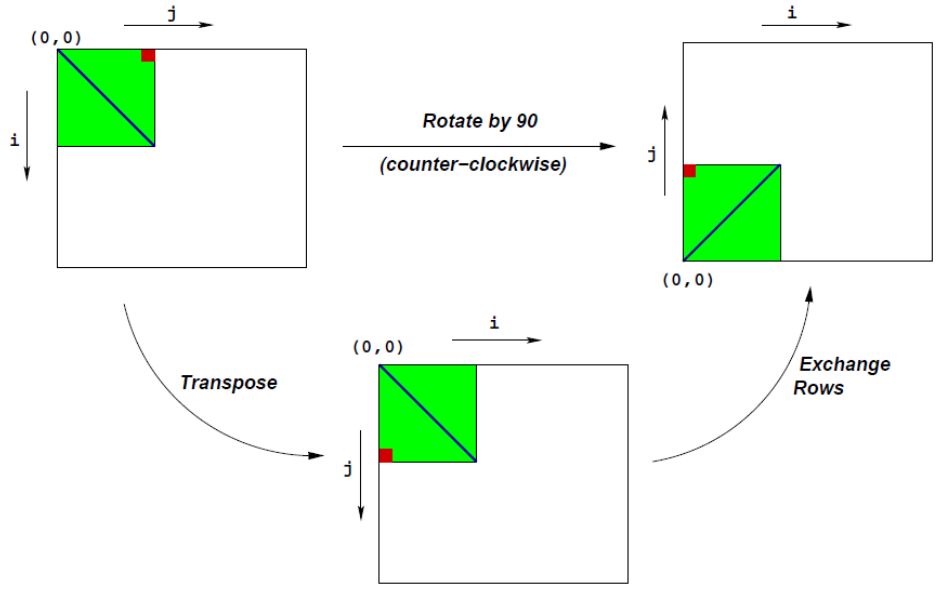

Figure 1 Rotation of an image by 90° counterclockwise

The **smooth** operation is implemented by replacing every pixel value with the average of all the pixels around it (in a maximum of 3×3 window centered at that pixel). Consider Figure 2. The values of pixels M2[1][1] and M2[N-1][N-1] are given below:

$$\mathtt{M2[1][1]} = \frac{\sum_{\mathtt{i}=0}^{2} \sum_{\mathtt{j}=0}^{2} \mathtt{M1[i][j]}}{9}$$

$$\mathtt{M2[N-1][N-1]} = \frac{\sum_{\mathtt{i}=N-2}^{N-1} \sum_{\mathtt{j}=N-2}^{N-1} \mathtt{M1[i][j]}}{4}$$

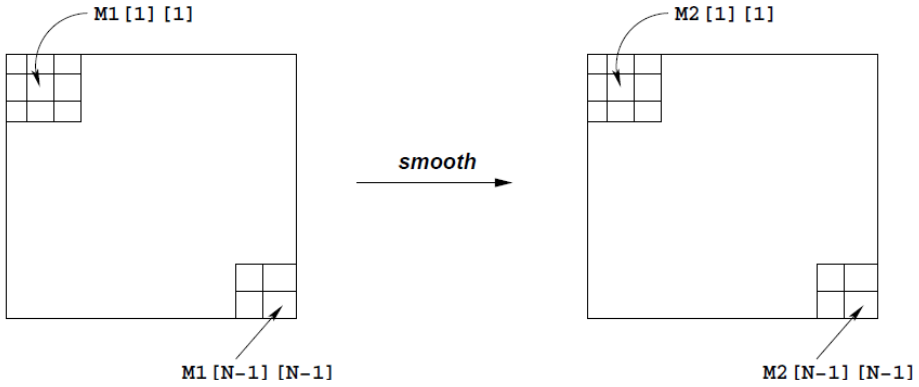

Figure 2 Smoothing an image

Source Code

The only file you will be modifying is **kernels.c**. Looking at the file **kernels.c**, you'll notice a C structure **team** into which you should insert the requested identifying information about the one or two individuals comprising your programming team.

(1) Data structures

The core data structure deals with image representation. A **pixel** is a struct as shown below:

```
typedef struct {
    unsigned short red;   /* R value */
    unsigned short green; /* G value */
    unsigned short blue;  /* B value */
} pixel;
```

As can be seen, RGB values have 16-bit representations ("16-bit color"). An image I is represented as a one-dimensional array of pixels, where the (i, j)th pixel is I[RIDX(i,j,n)]. Here n is the dimension of the image matrix, and RIDX is a macro defined as follows:

```
#define RIDX(i,j,n) ((i)*(n)+(j))
```

See the file **defs.h** for this code.

(2) Rotate operation

The following C function computes the result of rotating the source image **src** by 90° and stores the result in destination image **dst**. **dim** is the dimension of the image.

```
void naive_rotate(int dim, pixel *src, pixel *dst) {
  int i, j;

  for(i=0; i < dim; i++)
    for(j=0; j < dim; j++)
      dst[RIDX(dim-1-j,i,dim)] = src[RIDX(i,j,dim)];

  return;
}
```

The above code scans the rows of the source image matrix, copying to the columns of the destination image matrix. **(Activity 1)** Your task is to discuss the reasons of performance improvement from the functions rotate_c、rotate_b8、rotate_b4、rotate_b16_u2、rotate_b32_u2、rotate_b8_u4_c、rotate_b8_u4、rotate_b16_u4、rotate_b16_u4_c、rotate_b32_u4、rotate_b32_u4_c.

See the file **kernels.c** for this code.

(3) Smooth operation

The smoothing function takes as input a source image **src** and returns the smoothed result in the destination image **dst**. Here is part of an implementation:

```
void naive_smooth(int dim, pixel *src, pixel *dst) {
  int i, j;

  for(i=0; i < dim; i++)
    for(j=0; j < dim; j++)
      dst[RIDX(i,j,dim)] = avg(dim, i, j, src); /* Smooth the (i,j)th pixel */

  return;
}
```

The function avg returns the average of all the pixels around the (i,j)th pixel. **(Activity 2)** Your task is to optimize smooth (and avg) to run as fast as possible.

This code (and an implementation of avg) is in the file **kernels.c**.

(Activity 2) Your task is to discuss the reasons of performance improvement.

(4) Performance measures

Our main performance measure is **CPE (Cycles per Element)**. If a function takes C cycles to run for an image of size N×N, the CPE value is C/N^2. Performance is shown for 5 different values of N.

The ratios (speedups) of the optimized implementation over the naive one will constitute a score of your implementation. To summarize the overall effect over different values of N, we will compute the geometric mean of the results for these 5 values. That is, if the measured speedups for N = {32, 64, 128, 256, 512} are R32, R64, R128, R256, and R512 then we compute the overall performance as

$$R = \sqrt[5]{R_{32} \times R_{64} \times R_{128} \times R_{256} \times R_{512}}$$

Running

You should install the **gcc-multilib** to compile the source code

```
unix> sudo apt-get install gcc-multilib
```

The source code you will write will be linked with object code that we supply into a driver binary. To create this binary, you will need to execute the command

```
unix> make driver
```

You will need to re-make driver each time you change the code in **kernels.c**. To test your implementations, you can then run the command:

```
unix> ./driver
```

3. Lab Device or Environment

The lab is performed on an Ubuntu virtual machine (16.04 Xenial Xerus, 32-bit (i386) , desktop image installation), installed in VMWare Workstation 17.5.1 on a Windows 11 laptop. An OpenSSH server is set on the virtual machine, and operations are performed with a loopback SSH connection on Windows PowerShell.

The system information is listed as follows (fig. 1):

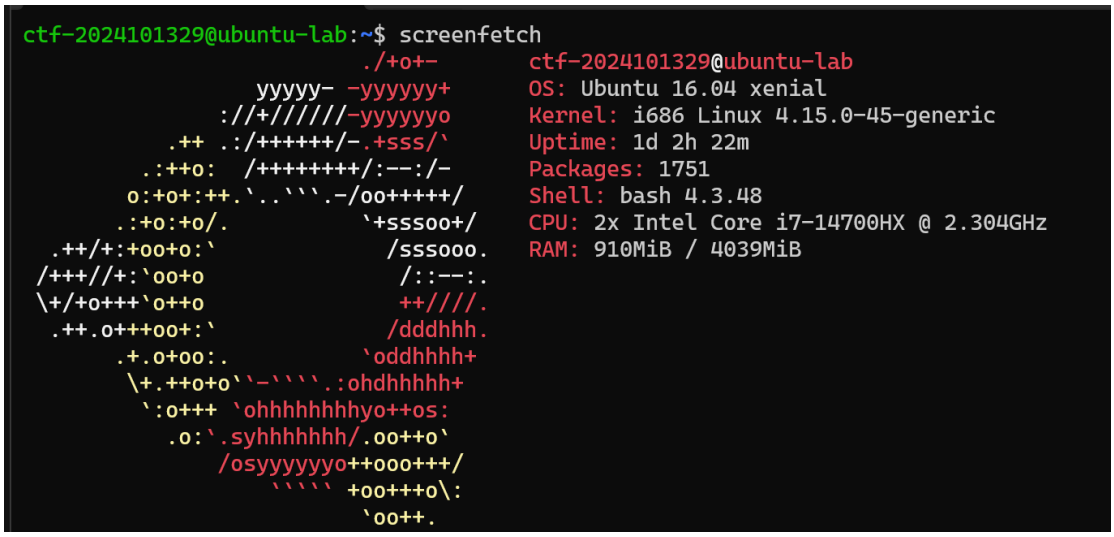

Figure 3 Screenshot of screenfetch command on the virtual machine

4. Results and Analysis

Preparations

We ran the driver program a few times (shows in figure 4, for example) to get the baseline CPE value for rotate and smooth functions, which is written in **config.h** for calibration:

```
ctf-2024101329@ubuntu-lab:~/Documents/computer-organization-labs/lab-6/perflab/perflab$ ./driver
Teamname: Nannimi
Member 1: Li Rongyao
Email 1: lirongyao6310@stu2024.jnu.edu.cn
Member 2: Chen Tuanfei
Email 2: terrychen@stu2024.jnu.edu.cn
Member 3: Yang Qicheng
Email 3: 15712100828@163.com

Rotate: Version = naive_rotate: Naive baseline implementation:
Dim             64      128     256     512     1024    Mean
Your CPEs       3.7     19.9    8.2     10.2    18.9
Baseline CPEs   7.0     14.3    13.6    14.3    15.3
Speedup         1.9     0.7     1.7     1.4     0.8     1.2

Rotate: Version = rotate_c: copying pixel by pixel, no loop unrolling, dst-sequential:
Dim             64      128     256     512     1024    Mean
Your CPEs       3.4     8.1     8.4     13.1    20.0
Baseline CPEs   7.0     14.3    13.6    14.3    15.3
Speedup         2.1     1.8     1.6     1.1     0.8     1.4

Rotate: Version = rotate_b4: copying by 4 x 4 blocks, no loop unrolling, src-sequential:
Dim             64      128     256     512     1024    Mean
Your CPEs       6.9     7.0     2.9     3.8     3.4
Baseline CPEs   7.0     14.3    13.6    14.3    15.3
Speedup         1.0     2.0     4.7     3.8     4.5     2.8
```

```
Rotate: Version = rotate_b4_c: copying by 4 x 4 blocks, no loop unrolling, dst-sequential:
Dim             64      128     256     512     1024    Mean
Your CPEs       2.9     2.8     4.3     2.8     6.7
Baseline CPEs   7.0     14.3    13.6    14.3    15.3
Speedup         2.4     5.1     3.2     5.1     2.3     3.4

Rotate: Version = rotate_b4_u2: copying by 4 x 4 blocks, 2x loop unrolling, src-sequential:
Dim             64      128     256     512     1024    Mean
Your CPEs       2.8     6.9     4.3     5.0     4.3
Baseline CPEs   7.0     14.3    13.6    14.3    15.3
Speedup         2.5     2.1     3.2     2.8     3.5     2.8

Rotate: Version = rotate_b4_u2_c: copying by 4 x 4 blocks, 2x loop unrolling, dst-sequential:
Dim             64      128     256     512     1024    Mean
Your CPEs       7.0     2.9     2.9     7.1     6.6
Baseline CPEs   7.0     14.3    13.6    14.3    15.3
Speedup         1.0     4.9     4.6     2.0     2.3     2.5

Rotate: Version = rotate_b4_u4: copying by 4 x 4 blocks, 4x loop unrolling, src-sequential:
Dim             64      128     256     512     1024    Mean
Your CPEs       5.7     2.5     6.7     2.9     4.4
Baseline CPEs   7.0     14.3    13.6    14.3    15.3
Speedup         1.2     5.6     2.0     5.0     3.5     3.0

Rotate: Version = rotate_b4_u4_c: copying by 4 x 4 blocks, 4x loop unrolling, dst-sequential:
Dim             64      128     256     512     1024    Mean
Your CPEs       7.3     7.2     3.0     5.0     8.7
Baseline CPEs   7.0     14.3    13.6    14.3    15.3
Speedup         1.0     2.0     4.6     2.9     1.8     2.1

Rotate: Version = rotate_b8: copying by 8 x 8 blocks, no loop unrolling, src-sequential:
Dim             64      128     256     512     1024    Mean
Your CPEs       1.3     4.9     5.1     2.5     3.9
Baseline CPEs   7.0     14.3    13.6    14.3    15.3
Speedup         5.6     2.9     2.7     5.6     3.9     3.9

Rotate: Version = rotate_b8_c: copying by 8 x 8 blocks, no loop unrolling, dst-sequential:
Dim             64      128     256     512     1024    Mean
Your CPEs       2.0     4.7     4.6     4.6     6.7
Baseline CPEs   7.0     14.3    13.6    14.3    15.3
Speedup         3.5     3.0     2.9     3.1     2.3     2.9

Rotate: Version = rotate_b8_u2: copying by 8 x 8 blocks, 2x loop unrolling, src-sequential:
Dim             64      128     256     512     1024    Mean
Your CPEs       4.7     5.1     5.4     5.6     5.3
Baseline CPEs   7.0     14.3    13.6    14.3    15.3
Speedup         1.5     2.8     2.5     2.6     2.9     2.4

Rotate: Version = rotate_b8_u2_c: copying by 8 x 8 blocks, 2x loop unrolling, dst-sequential:
Dim             64      128     256     512     1024    Mean
Your CPEs       4.2     1.9     4.5     2.0     6.4
Baseline CPEs   7.0     14.3    13.6    14.3    15.3
Speedup         1.7     7.6     3.0     7.1     2.4     3.6

Rotate: Version = rotate_b8_u4: copying by 8 x 8 blocks, 4x loop unrolling, src-sequential:
Dim             64      128     256     512     1024    Mean
Your CPEs       1.8     1.9     4.8     3.9     3.6
Baseline CPEs   7.0     14.3    13.6    14.3    15.3
Speedup         4.0     7.5     2.8     3.7     4.3     4.2
```

```
Rotate: Version = rotate_b8_u4_c: copying by 8 x 8 blocks, 4x loop unrolling, dst-sequential:
Dim             64      128     256     512     1024    Mean
Your CPEs       2.3     4.7     4.6     4.9     3.9
Baseline CPEs   7.0     14.3    13.6    14.3    15.3
Speedup         3.0     3.0     2.9     2.9     3.9     3.1

Rotate: Version = rotate_b16: copying by 16 x 16 blocks, no loop unrolling, src-sequential:
Dim             64      128     256     512     1024    Mean
Your CPEs       2.5     2.5     2.5     2.9     6.8
Baseline CPEs   7.0     14.3    13.6    14.3    15.3
Speedup         2.8     5.8     5.4     5.0     2.2     4.0

Rotate: Version = rotate_b16_c: copying by 16 x 16 blocks, no loop unrolling, dst-sequential:
Dim             64      128     256     512     1024    Mean
Your CPEs       2.4     2.3     5.3     5.8     6.6
Baseline CPEs   7.0     14.3    13.6    14.3    15.3
Speedup         2.9     6.2     2.6     2.5     2.3     3.1

Rotate: Version = rotate_b16_u2: copying by 16 x 16 blocks, 2x loop unrolling, src-sequential:
Dim             64      128     256     512     1024    Mean
Your CPEs       4.2     4.4     5.8     3.5     4.7
Baseline CPEs   7.0     14.3    13.6    14.3    15.3
Speedup         1.7     3.2     2.3     4.0     3.2     2.8
```

```
Rotate: Version = rotate_b16_u2_c: copying by 16 x 16 blocks, 2x loop unrolling, dst-sequential:
Dim             64      128     256     512     1024    Mean
Your CPEs       1.7     1.7     1.7     1.9     2.2
Baseline CPEs   7.0     14.3    13.6    14.3    15.3
Speedup         4.0     8.4     8.2     7.6     6.9     6.8

Rotate: Version = rotate_b16_u4: copying by 16 x 16 blocks, 4x loop unrolling, src-sequential:
Dim             64      128     256     512     1024    Mean
Your CPEs       3.7     4.1     3.2     3.0     7.1
Baseline CPEs   7.0     14.3    13.6    14.3    15.3
Speedup         1.9     3.5     4.3     4.8     2.2     3.1

Rotate: Version = rotate_b16_u4_c: copying by 16 x 16 blocks, 4x loop unrolling, dst-sequential:
Dim             64      128     256     512     1024    Mean
Your CPEs       4.1     3.8     3.9     1.9     2.8
Baseline CPEs   7.0     14.3    13.6    14.3    15.3
Speedup         1.7     3.8     3.5     7.5     5.4     3.9

Rotate: Version = rotate_b32: copying by 32 x 32 blocks, no loop unrolling, src-sequential:
Dim             64      128     256     512     1024    Mean
Your CPEs       2.3     2.7     7.0     6.2     9.9
Baseline CPEs   7.0     14.3    13.6    14.3    15.3
Speedup         3.1     5.2     1.9     2.3     1.5     2.6

Rotate: Version = rotate_b32_c: copying by 32 x 32 blocks, no loop unrolling, dst-sequential:
Dim             64      128     256     512     1024    Mean
Your CPEs       2.0     4.8     2.0     5.0     2.7
Baseline CPEs   7.0     14.3    13.6    14.3    15.3
Speedup         3.4     3.0     6.8     2.9     5.6     4.1

Rotate: Version = rotate_b32_u2: copying by 32 x 32 blocks, 2x loop unrolling, src-sequential:
Dim             64      128     256     512     1024    Mean
Your CPEs       1.5     5.7     4.2     5.1     6.5
Baseline CPEs   7.0     14.3    13.6    14.3    15.3
Speedup         4.5     2.5     3.2     2.8     2.4     3.0

Rotate: Version = rotate_b32_u2_c: copying by 32 x 32 blocks, 2x loop unrolling, dst-sequential:
Dim             64      128     256     512     1024    Mean
Your CPEs       1.5     1.5     1.4     1.8     2.8
Baseline CPEs   7.0     14.3    13.6    14.3    15.3
Speedup         4.8     9.4     9.5     7.8     5.4     7.1

Rotate: Version = rotate_b32_u4: copying by 32 x 32 blocks, 4x loop unrolling, src-sequential:
Dim             64      128     256     512     1024    Mean
Your CPEs       1.5     4.3     1.7     2.6     6.7
Baseline CPEs   7.0     14.3    13.6    14.3    15.3
Speedup         4.7     3.4     8.2     5.4     2.3     4.4

Rotate: Version = rotate_b32_u4_c: copying by 32 x 32 blocks, 4x loop unrolling, dst-sequential:
Dim             64      128     256     512     1024    Mean
Your CPEs       1.5     3.5     3.5     4.0     3.0
Baseline CPEs   7.0     14.3    13.6    14.3    15.3
Speedup         4.6     4.1     3.9     3.6     5.1     4.2
```

```
Smooth: Version = naive_smooth: Naive baseline implementation:
Dim             32      64      128     256     512     Mean
Your CPEs       79.3    32.7    32.6    32.7    33.3
Baseline CPEs   63.5    62.1    51.1    47.6    56.4
Speedup         0.8     1.9     1.6     1.5     1.7     1.4

Smooth: Version = Copying the whole matrix sequentially for every pixel in kernel:
Dim             32      64      128     256     512     Mean
Your CPEs       104.3   109.6   95.0    47.3    55.3
Baseline CPEs   63.5    62.1    51.1    47.6    56.4
Speedup         0.6     0.6     0.5     1.0     1.0     0.7

Smooth: Version = Added three row buffers and sliding window calculation to prevent excessive arithmetic and memory access:
Dim             32      64      128     256     512     Mean
Your CPEs       52.0    22.0    51.5    25.0    24.3
Baseline CPEs   63.5    62.1    51.1    47.6    56.4
Speedup         1.2     2.8     1.0     1.9     2.3     1.7

Smooth: Version = Expanded function calls for better pipeline optimization:
Dim             32      64      128     256     512     Mean
Your CPEs       51.1    23.9    51.4    21.9    31.5
Baseline CPEs   63.5    62.1    51.1    47.6    56.4
Speedup         1.2     2.6     1.0     2.2     1.8     1.7

Summary of Your Best Scores:
  Rotate: 7.1 (rotate_b32_u2_c: copying by 32 x 32 blocks, 2x loop unrolling, dst-sequential)
  Smooth: 1.7 (Added three row buffers and sliding window calculation to prevent excessive arithmetic and memory access)
```

Figure 4 Screenshot of “driver” program output (baseline CPE calibrated)

By repeatedly running the **driver** program, our team discovered that the CPE measurement can be very unstable, and interestingly, the instability is closely related to the average value. In order to

present more accurate measurements, we have to run the **driver** program for many times and give conclusions with regards to the average result.

To implement this, our team wrote a Python script named **multiple-test.py**. The script automatically recompiles the program and repeatedly runs the **driver** program, whose output is redirected to append a text file named **results**. After each run, the file reader parses the lines in appended output into dictionary entries with function name associated with its speedup factor, and then writes it into a CSV file named **performance_raw_data.csv**. This CSV file can then be used to present statistical results and visualizations.

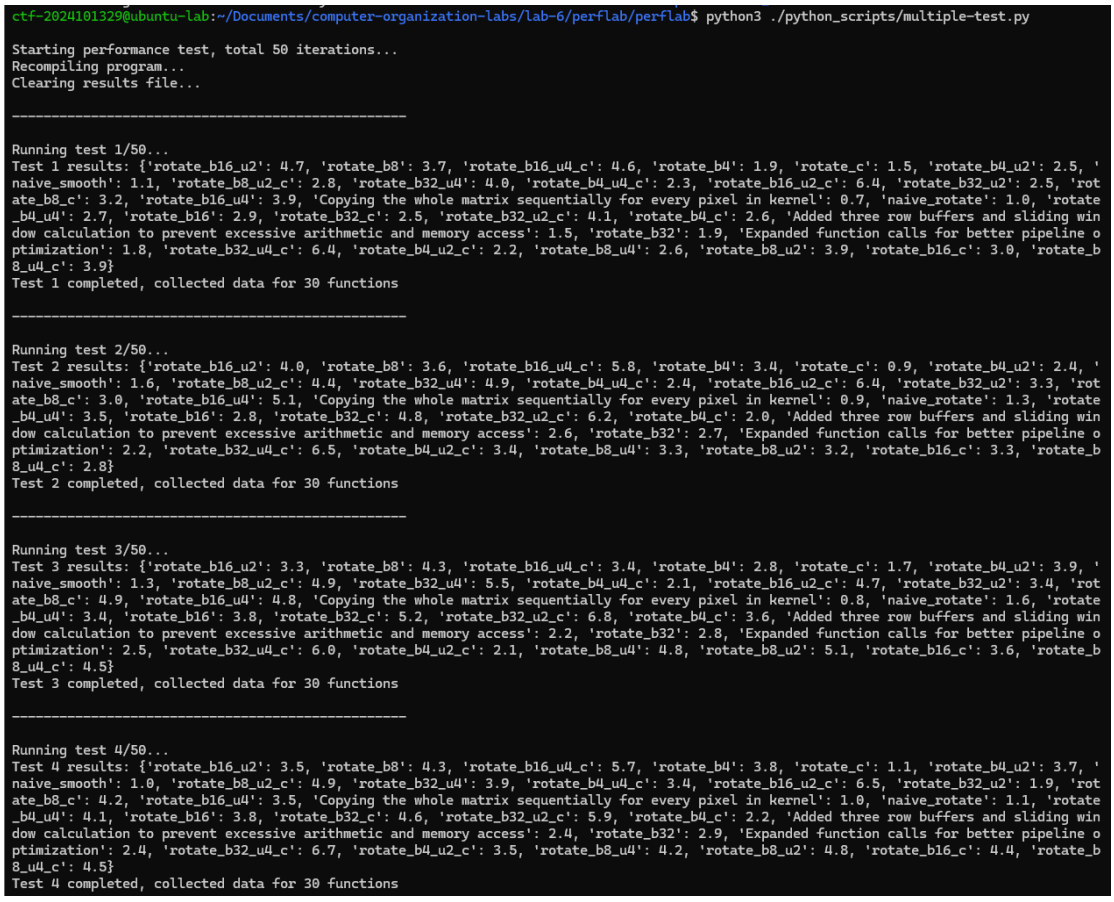

Figure 5 Screenshot of **multiple-test.py** output (truncated)

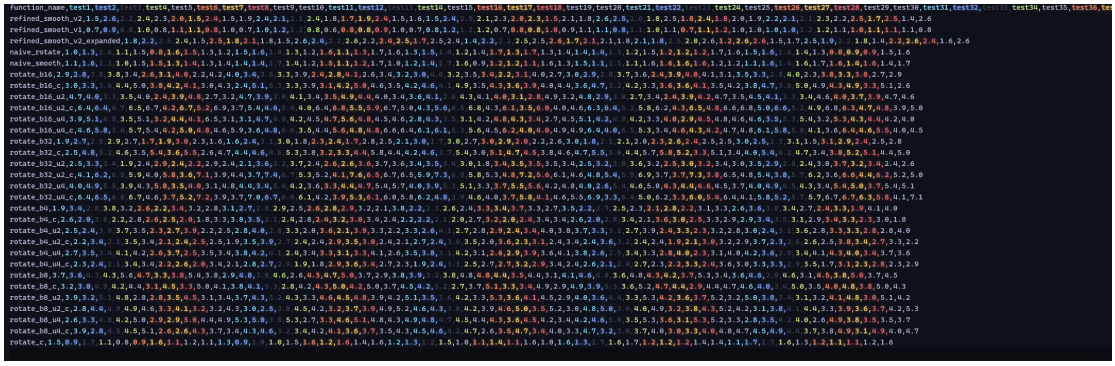

Figure 6 Screenshot of **performance_raw_data.csv** (50 runs)

The test result (speedup factor of rotate and smooth implementations) with 50 runs of **driver** program is visualized as the following diagram:

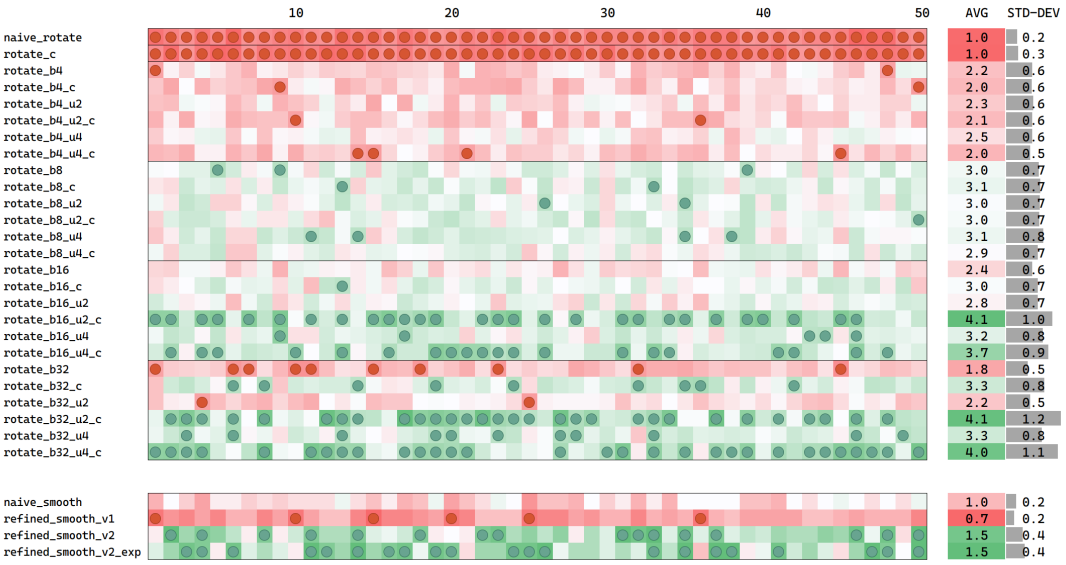

Figure 7 Test results of rotate and smooth implementations (speedup factor relative to baseline)

Each row corresponds to a function, and each narrow column corresponds to one test by running the **driver** program. The two wide columns on the right are the average values and standard deviations of speedup factor. A cell is colored red or green by whether its value is below or above the median of all values (respective for rotate/smooth), and cells with bottom 10% and top 90% are marked with red or green dots.

Activity 1

(Your task is to discuss the reasons of performance improvement from the functions rotate_c、rotate_b8、rotate_b4、rotate_b16_u2、rotate_b32_u2、rotate_b8_u4_c、rotate_b8_u4、rotate_b16_u4、rotate_b16_u4_c、rotate_b32_u4、rotate_b32_u4_c.)

Discussion

Rotation function performs the action of copying pixel values one by one, from one array to another. The naive implementation is to load from the source array in a row-major (sequential) pattern, which causes the destination array to be written into in a column-major pattern. This results in the worst performance, which is also our baseline of speedup measurement.

There are three modifications that can potentially bring better performance to rotate implementation, which we will model as vector **(b, u, c)**: Copying in **b*b** blocks instead of sequential pixels, unrolling the innermost loop by a factor of **u**, and swapping the access pattern of source array and destination array (**c**=1 means to swap). Instead of merely assessing the test cases mentioned in requirements, we let **b=[1,4,8,16,32]**, **u=[1,2,4]**, **c=[0,1]**, and test on every case in the Cartesian product, omitting cases in which b=1 and u≠1 (since such unrolling cannot be written in a unified logic like the block-copying ones). This covers the required cases, while giving us a more complete and systematic view on how these factors affect the performance.

We first look at the two problem-specific modification: Copying in blocks instead of lines (**b**), and swapping the access patterns of source and destinations (**c**). That is to say, they change the access pattern in different ways for better temporal and spatial locality, aiming to minimize the penalty of cache misses.

The problem with column major pattern is that it is stride-**N**, and every time it accesses an element, it will not return to a nearby position in the future **N** accesses. Therefore, as **N** gets sufficiently large, it is reasonable to foresee that **N** is too large that a block placed on the cache with recently accessed data can be reused neither immediately (poor spatial locality) nor before the data object is overwritten because of limited capacity (poor temporal locality). As a result, there will be a level in L1-L3 where cache miss will happen on every access.

Copying in blocks resolves the problem by improving the column-major locality at the tradeoff of row-major locality, leading to net positive performance improvement. As shown in the following figure, by setting a block size, a block (or even multiple consecutive blocks) can be fully loaded into the cache in the first few cache misses, and future cache misses will not happen until the blocks have been fully accessed. Notice that this only enhances temporal locality but not spatial locality, because with blocks or not, the array is still accessed in a stride-**N** pattern.

While block-partitioned accessing reduces the frequency of cache misses, swapping accessing patterns for source array and destination array aims to exchange the amount of write latency and read latency due to cache misses, since in typical memory systems, the former is generally more costly than the latter.

Another optimization is loop-unrolling, which reduces the number of conditional jumps in loops by processing multiple elements in a single iteration. Since these operations are free of control dependency caused by conditional jumps, they can be well pipelined, improving the throughput of element processing.

Result Analysis

According to the test results, as the block size goes larger and larger, the average performance regardless of other configurations gets better, and the difference of applying pattern swapping or not becomes more and more obvious.

Without pattern swapping (read-sequential), the speedup factor reaches a bottle neck at 3.3x, and does not always get better as the block size increases. The bottleneck marks the peak of moderately large blocks (b=8 and b=16), whose performance is highly dependent on the loop unrolling factor, and cannot reach this bottleneck unless the loop unrolling factor is set to the maximum u=4. It shows that the penalty of cache misses in write access is not only costly, but also difficult to optimize.

On the other hand, the result with pattern swapping (write-sequential) looks more promising. A steady improvement can be observed as the block size doubles again and again, despite that marginal improvement diminishes from b=16 to b=32. Loop unrolling also gives certain performance improvement, but is not as obvious as the read-sequential case. We speculate that this may be due to CPU's different strategies of detecting data dependencies of pipelining read and write access in memory.

In conclusion, to make the rotation function optimal with these 3 modifications, the access pattern should be set to write-sequential and partitioned by blocks with appropriate size (depending on the cache capacity). With such configuration, unrolling the loop within iteration of each block gives a certain impact, but does not play a decisive role.

Activity 2

(Your task is to optimize smooth (and avg) to run as fast as possible. Your task is to discuss the reasons of performance improvement.)

Discussion (refined_smooth_v1)

The smooth function calculates a convolution of a given path with a 3 by 3 averaging kernel. The naive implementation sequentially iterates over the source array. For each pixel, it adds the value of surrounding pixels and itself (9 pixels in total, conditionally omitting the ones that are out of the array's range), and divide the sum by the number of pixels accumulated. This is the average pixel, which will be written into the destination array.

Our team first noticed that the calculation of every pixel requires retrieving data from different rows, which is distant from each other, resulting in an access pattern with poor spatial locality. Therefore, our first thought is to make the source array read sequentially as well. We designed the function **refined_smooth_v1**, which declares an array of pixel sums, and instead of retrieving the 9 nearby pixels immediately for each single pixel, we consider the resulting average image as the stack of 9 images, each with a position shift by one of the Cartesian product $\{-1, 0, 1\} \times \{-1, 0, 1\}$. The refined smooth algorithm adds the 9 shifted to the sum one by one, so that the pattern of adding pixels in each graph is sequential for both reading from source and writing in destination, maintaining a good spatial locality and temporal locality.

Result Analysis (refined_smooth_v1)

However, as the result source, this turns out to be a negative optimization that degrades the performance (with a poor 9.9 speedup factor, compared to the 13.9 of naive one). We speculate that this is due to excessive access to the sum array (which plays the role of temporary sum pixel in naive algorithm), in which each pixel is written into 9 times. The tradeoff of excessive operation outweighs the benefit of maintaining cache-friendly locality, which itself, by Amdahl's law, does not have much space for improvement.

Discussion (refined_smooth_v2, refined_smooth_v2_expanded)

Observing that excessive operation (e.g. reading/writing in the same pixel repeatedly) is the major problem, our team designed the function **refined_smooth_v2**, which separates the calculation of horizontal sum (sequential) and vertical sum (non-sequential), and uses a "sliding window" technique when computing the horizontal sums pixel by pixel.

"Sliding window" refers to the technique to optimize the computing of a series of values using a recurrence relation—in which every next value comes from the previous value—instead of calculating each value independently. This saves quite a few operations in certain cases, such as adding three adjacent pixels, which now can be calculated by subtracting a pixel from the previous sum and then adding in a new one.

By “separating the calculation of horizontal sum and vertical sum”, we mean to calculate the sum of a pixel and its left, right pixels as a group of three for each row. The sum of a row will then be stored into a row buffer. A row’s final result can then be calculated as the sum of three adjacent row sums. Instead of using a whole matrix of pixel sums like the previous algorithm, we only need to use three row buffers this time, which can certainly be fit in a cache and save a lot of read/write latency. The calculation of each row overwrites one row buffer with the new row sum, and by averaging the three row buffers in a unified process, we can get the final sum of a row.

We also expanded the function calls as **refined_smooth_v2_expanded**, hoping to improve the performance to some extent by reducing the times control-dependent jumps。

Result analysis (refined_smooth_v2, refined_smooth_v2_expanded)

It turns out that the new algorithm design has a much better performance, with a speedup factor of 1.5x compared to the naive algorithm. Expanding the function calls leads to a slight improvement, which we consider is not worth it at the tradeoff of seriously degrading the code’s readability.

5. Source Code

config.h

```
/********************************************************
 * config.h - Configuration data for the driver.c program.
 ********************************************************/
#ifndef _CONFIG_H_
#define _CONFIG_H_

/*
 * CPEs for the baseline (naive) version of the rotate function that
 * was handed out to the students. Rd is the measured CPE for a dxd
 * image. Run the driver.c program on your system to get these
 * numbers.
 */
#define R64   7.0
#define R128  14.3
#define R256  13.6
#define R512  14.3
#define R1024 15.3

/*
 * CPEs for the baseline (naive) version of the smooth function that
 * was handed out to the students. Sd is the measure CPE for a dxd
 * image. Run the driver.c program on your system to get these
 * numbers.
 */
#define S32   63.5
#define S64   62.1
#define S128  51.1
#define S256  47.6
#define S512  56.4


#endif /* _CONFIG_H_ */
```

kernels.c

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include "defs.h"

/*
 * Please fill in the following team struct
 */
team_t team = {
    "Nannimi", /* Team name */

    "Li Rongyao",                    /* First member full name */
    "lirongyao6310@stu2024.jnu.edu.cn", /* First member email address */

    "Chen Tuanfei",  /* Second or third member full name (leave blank if none) */
    "terrychen@stu2024.jnu.edu.cn", /* Second or third member email addr (leave blank if none) */

    "Yang Qicheng",
    "15712100828@163.com"

};

/***************
 * ROTATE KERNEL
 ***************/

/******************************************************
 * Your different versions of the rotate kernel go here
 ******************************************************/

/*
 * naive_rotate - The naive baseline version of rotate
 */
char naive_rotate_descr[] = "naive_rotate: Naive baseline implementation";
void naive_rotate(int dim, pixel *src, pixel *dst)
{
    int i, j;

    for (i = 0; i < dim; i++)
    for (j = 0; j < dim; j++)
        dst[RIDX(dim-1-j, i, dim)] = src[RIDX(i, j, dim)];
}

/*
 * rotate - Your current working version of rotate
 * IMPORTANT: This is the version you will be graded on
 */
char rotate_descr[] = "rotate: Current working version";
void rotate(int dim, pixel *src, pixel *dst)
{

}
/*
 * process_block_u1 - Inner loop processing function for u=1 (no unrolling)
 */
static void process_block_u1(int dim, pixel *src, pixel *dst, int b_size,
                             int bi_start, int bj_start, int c) {
    if (c == 0) {
        // row-major order (bi outer, bj inner)
        for (int bi = bi_start; bi < bi_start + b_size; bi++) {
            for (int bj = bj_start; bj < bj_start + b_size; bj++) {
                dst[RIDX(dim - 1 - bj, bi, dim)] = src[RIDX(bi, bj, dim)];
            }
```

```
        }
    } else {
        // column-major order (bj outer, bi inner)
        for (int bj = bj_start; bj < bj_start + b_size; bj++) {
            for (int bi = bi_start; bi < bi_start + b_size; bi++) {
                dst[RIDX(dim - 1 - bj, bi, dim)] = src[RIDX(bi, bj, dim)];
            }
        }
    }
}

/*
 * process_block_u2 - Inner loop processing function for u=2 (2x unrolling)
 */
static void process_block_u2(int dim, pixel *src, pixel *dst, int b_size,
                             int bi_start, int bj_start, int c) {
    if (c == 0) {
        // row-major order (bi outer, bj inner)
        for (int bi = bi_start; bi < bi_start + b_size; bi += 2) {
            for (int bj = bj_start; bj < bj_start + b_size; bj++) {
                dst[RIDX(dim - 1 - bj, bi, dim)] = src[RIDX(bi, bj, dim)];
                dst[RIDX(dim - 1 - bj, bi + 1, dim)] = src[RIDX(bi + 1, bj, dim)];
            }
        }
    } else {
        // column-major order (bj outer, bi inner)
        for (int bj = bj_start; bj < bj_start + b_size; bj++) {
            for (int bi = bi_start; bi < bi_start + b_size; bi += 2) {
                dst[RIDX(dim - 1 - bj, bi, dim)] = src[RIDX(bi, bj, dim)];
                dst[RIDX(dim - 1 - bj, bi + 1, dim)] = src[RIDX(bi + 1, bj, dim)];
            }
        }
    }
}

/*
 * process_block_u4 - Inner loop processing function for u=4 (4x unrolling)
 */
static void process_block_u4(int dim, pixel *src, pixel *dst, int b_size,
                             int bi_start, int bj_start, int c) {
    if (c == 0) {
        // row-major order (bi outer, bj inner)
        for (int bi = bi_start; bi < bi_start + b_size; bi += 4) {
            for (int bj = bj_start; bj < bj_start + b_size; bj++) {
                dst[RIDX(dim - 1 - bj, bi, dim)] = src[RIDX(bi, bj, dim)];
                dst[RIDX(dim - 1 - bj, bi + 1, dim)] = src[RIDX(bi + 1, bj, dim)];
                dst[RIDX(dim - 1 - bj, bi + 2, dim)] = src[RIDX(bi + 2, bj, dim)];
                dst[RIDX(dim - 1 - bj, bi + 3, dim)] = src[RIDX(bi + 3, bj, dim)];
            }
        }
    } else {
        // column-major order (bj outer, bi inner)
        for (int bj = bj_start; bj < bj_start + b_size; bj++) {
            for (int bi = bi_start; bi < bi_start + b_size; bi += 4) {
                dst[RIDX(dim - 1 - bj, bi, dim)] = src[RIDX(bi, bj, dim)];
                dst[RIDX(dim - 1 - bj, bi + 1, dim)] = src[RIDX(bi + 1, bj, dim)];
                dst[RIDX(dim - 1 - bj, bi + 2, dim)] = src[RIDX(bi + 2, bj, dim)];
                dst[RIDX(dim - 1 - bj, bi + 3, dim)] = src[RIDX(bi + 3, bj, dim)];
            }
        }
    }
}
```

```c
/*
 * rotate_unified - Unified rotate function with parameters b, u, c
 */
void rotate_unified(int dim, pixel *src, pixel *dst, int b, int u, int c)
{
    if (c == 0) {
        // i-j loop order
        for (int i = 0; i < dim; i += b) {
            for (int j = 0; j < dim; j += b) {
                if (u == 1) process_block_u1(dim, src, dst, b, j, i, c);
                else if (u == 2) process_block_u2(dim, src, dst, b, j, i, c);
                else if (u == 4) process_block_u4(dim, src, dst, b, j, i, c);
            }
        }
    } else {
        // j-i loop order (optimized for memory access)
        for (int j = 0; j < dim; j += b) {
            for (int i = 0; i < dim; i += b) {
                if (u == 1) process_block_u1(dim, src, dst, b, j, i, c);
                else if (u == 2) process_block_u2(dim, src, dst, b, j, i, c);
                else if (u == 4) process_block_u4(dim, src, dst, b, j, i, c);
            }
        }
    }
}

/*
 * rotate_c - Optimize the loop sequence to dst-sequential
 */
char rotate_c_descr[] = "rotate_c: copying pixel by pixel, no loop unrolling, dst-sequential";
void rotate_c(int dim, pixel *src, pixel *dst)
{
    int i, j;

    for (i = 0; i < dim; i++)
    for (j = 0; j < dim; j++)
        dst[RIDX(dim-1-j, i, dim)] = src[RIDX(i, j, dim)];
}



/*
 * rotate_b8 - copying by 8 x 8 blocks, unrolling loop with 1 iterations at a time, src-sequential
 */

char rotate_b8_descr[] = "rotate_b8: copying by 8 x 8 blocks, no loop unrolling, src-sequential";
void rotate_b8(int dim, pixel *src, pixel *dst)
{
    rotate_unified(dim, src, dst, 8, 1, 0);
}

/*
 * rotate_b4 - copying by 4 x 4 blocks, no loop unrolling, src-sequential
 */

char rotate_b4_descr[] = "rotate_b4: copying by 4 x 4 blocks, no loop unrolling, src-sequential";
void rotate_b4(int dim, pixel *src, pixel *dst)
{
    rotate_unified(dim, src, dst, 4, 1, 0);
}

/*
 * rotate_b16_u2 - copying by 16 x 16 blocks, 2x loop unrolling, src-sequential
 */
```

```
char rotate_b16_u2_descr[] = "rotate_b16_u2: copying by 16 x 16 blocks, 2x loop unrolling,
src-sequential";
void rotate_b16_u2(int dim, pixel *src, pixel *dst)
{
    rotate_unified(dim, src, dst, 16, 2, 0);
}

/*
 * rotate_b32_u2 - copying by 32 x 32 blocks, 2x loop unrolling, src-sequential
 */

char rotate_b32_u2_descr[] = "rotate_b32_u2: copying by 32 x 32 blocks, 2x loop unrolling,
src-sequential";
void rotate_b32_u2(int dim, pixel *src, pixel *dst)
{
    rotate_unified(dim, src, dst, 32, 2, 0);
}

/*
 * rotate_b8_u4 - copying by 8 x 8 blocks, 4x loop unrolling, src-sequential
 */

char rotate_b8_u4_descr[] = "rotate_b8_u4: copying by 8 x 8 blocks, 4x loop unrolling,
src-sequential";
void rotate_b8_u4(int dim, pixel *src, pixel *dst)
{
    rotate_unified(dim, src, dst, 8, 4, 0);
}

/*
 * rotate_b8_u4_c - copying by 8 x 8 blocks, 4x loop unrolling, dst-sequential
 */

char rotate_b8_u4_c_descr[] = "rotate_b8_u4_c: copying by 8 x 8 blocks, 4x loop unrolling,
dst-sequential";
void rotate_b8_u4_c(int dim, pixel *src, pixel *dst)
{
    rotate_unified(dim, src, dst, 8, 4, 1);
}

/*
 * rotate_b16_u4 - copying by 16 x 16 blocks, 4x loop unrolling, src-sequential
 */

char rotate_b16_u4_descr[] = "rotate_b16_u4: copying by 16 x 16 blocks, 4x loop unrolling,
src-sequential";
void rotate_b16_u4(int dim, pixel *src, pixel *dst)
{
    rotate_unified(dim, src, dst, 16, 4, 0);
}

/*
 * rotate_b16_u4_c - copying by 16 x 16 blocks, 4x loop unrolling, dst-sequential
 */

char rotate_b16_u4_c_descr[] = "rotate_b16_u4_c: copying by 16 x 16 blocks, 4x loop unrolling,
dst-sequential";
void rotate_b16_u4_c(int dim, pixel *src, pixel *dst)
{
    rotate_unified(dim, src, dst, 16, 4, 1);
}
```

```
/*
 * rotate_b32_u4 - copying by 32 x 32 blocks, 4x loop unrolling, src-sequential
 */

char rotate_b32_u4_descr[] = "rotate_b32_u4: copying by 32 x 32 blocks, 4x loop unrolling,
src-sequential";
void rotate_b32_u4(int dim, pixel *src, pixel *dst)
{
    rotate_unified(dim, src, dst, 32, 4, 0);
}

/*
 * rotate_b32_u4_c - copying by 32 x 32 blocks, 4x loop unrolling, dst-sequential
 */

char rotate_b32_u4_c_descr[] = "rotate_b32_u4_c: copying by 32 x 32 blocks, 4x loop unrolling,
dst-sequential";
void rotate_b32_u4_c(int dim, pixel *src, pixel *dst)
{
    rotate_unified(dim, src, dst, 32, 4, 1);
}

/*
 * rotate_b4_u2 - copying by 4 x 4 blocks, 2x loop unrolling, src-sequential
 */

char rotate_b4_u2_descr[] = "rotate_b4_u2: copying by 4 x 4 blocks, 2x loop unrolling,
src-sequential";
void rotate_b4_u2(int dim, pixel *src, pixel *dst)
{
    rotate_unified(dim, src, dst, 4, 2, 0);
}

/*
 * rotate_b4_u2_c - copying by 4 x 4 blocks, 2x loop unrolling, dst-sequential
 */

char rotate_b4_u2_c_descr[] = "rotate_b4_u2_c: copying by 4 x 4 blocks, 2x loop unrolling,
dst-sequential";
void rotate_b4_u2_c(int dim, pixel *src, pixel *dst)
{
    rotate_unified(dim, src, dst, 4, 2, 1);
}

/*
 * rotate_b4_u4 - copying by 4 x 4 blocks, 4x loop unrolling, src-sequential
 */

char rotate_b4_u4_descr[] = "rotate_b4_u4: copying by 4 x 4 blocks, 4x loop unrolling,
src-sequential";
void rotate_b4_u4(int dim, pixel *src, pixel *dst)
{
    rotate_unified(dim, src, dst, 4, 4, 0);
}

/*
 * rotate_b4_u4_c - copying by 4 x 4 blocks, 4x loop unrolling, dst-sequential
 */

char rotate_b4_u4_c_descr[] = "rotate_b4_u4_c: copying by 4 x 4 blocks, 4x loop unrolling,
dst-sequential";
void rotate_b4_u4_c(int dim, pixel *src, pixel *dst)
{
```

```
    rotate_unified(dim, src, dst, 4, 4, 1);
}

/*
 * rotate_b8_u2 - copying by 8 x 8 blocks, 2x loop unrolling, src-sequential
 */

char rotate_b8_u2_descr[] = "rotate_b8_u2: copying by 8 x 8 blocks, 2x loop unrolling,
src-sequential";
void rotate_b8_u2(int dim, pixel *src, pixel *dst)
{
    rotate_unified(dim, src, dst, 8, 2, 0);
}

/*
 * rotate_b8_u2_c - copying by 8 x 8 blocks, 2x loop unrolling, dst-sequential
 */

char rotate_b8_u2_c_descr[] = "rotate_b8_u2_c: copying by 8 x 8 blocks, 2x loop unrolling,
dst-sequential";
void rotate_b8_u2_c(int dim, pixel *src, pixel *dst)
{
    rotate_unified(dim, src, dst, 8, 2, 1);
}

/*
 * rotate_b16 - copying by 16 x 16 blocks, no loop unrolling, src-sequential
 */

char rotate_b16_descr[] = "rotate_b16: copying by 16 x 16 blocks, no loop unrolling, src-sequential";
void rotate_b16(int dim, pixel *src, pixel *dst)
{
    rotate_unified(dim, src, dst, 16, 1, 0);
}

/*
 * rotate_b16_c - copying by 16 x 16 blocks, no loop unrolling, dst-sequential
 */

char rotate_b16_c_descr[] = "rotate_b16_c: copying by 16 x 16 blocks, no loop unrolling,
dst-sequential";
void rotate_b16_c(int dim, pixel *src, pixel *dst)
{
    rotate_unified(dim, src, dst, 16, 1, 1);
}

/*
 * rotate_b32 - copying by 32 x 32 blocks, no loop unrolling, src-sequential
 */

char rotate_b32_descr[] = "rotate_b32: copying by 32 x 32 blocks, no loop unrolling, src-sequential";
void rotate_b32(int dim, pixel *src, pixel *dst)
{
    rotate_unified(dim, src, dst, 32, 1, 0);
}

/*
 * rotate_b32_c - copying by 32 x 32 blocks, no loop unrolling, dst-sequential
 */

char rotate_b32_c_descr[] = "rotate_b32_c: copying by 32 x 32 blocks, no loop unrolling,
dst-sequential";
void rotate_b32_c(int dim, pixel *src, pixel *dst)
```

```
{
    rotate_unified(dim, src, dst, 32, 1, 1);
}

/*
 * rotate_b4_c - copying by 4 x 4 blocks, no loop unrolling, dst-sequential
 */

char rotate_b4_c_descr[] = "rotate_b4_c: copying by 4 x 4 blocks, no loop unrolling, dst-sequential";
void rotate_b4_c(int dim, pixel *src, pixel *dst)
{
    rotate_unified(dim, src, dst, 4, 1, 1);
}

/*
 * rotate_b8_c - copying by 8 x 8 blocks, no loop unrolling, dst-sequential
 */

char rotate_b8_c_descr[] = "rotate_b8_c: copying by 8 x 8 blocks, no loop unrolling, dst-sequential";
void rotate_b8_c(int dim, pixel *src, pixel *dst)
{
    rotate_unified(dim, src, dst, 8, 1, 1);
}

/*
 * rotate_b16_u2_c - copying by 16 x 16 blocks, 2x loop unrolling, dst-sequential
 */

char rotate_b16_u2_c_descr[] = "rotate_b16_u2_c: copying by 16 x 16 blocks, 2x loop unrolling,
dst-sequential";
void rotate_b16_u2_c(int dim, pixel *src, pixel *dst)
{
    rotate_unified(dim, src, dst, 16, 2, 1);
}

/*
 * rotate_b32_u2_c - copying by 32 x 32 blocks, 2x loop unrolling, dst-sequential
 */

char rotate_b32_u2_c_descr[] = "rotate_b32_u2_c: copying by 32 x 32 blocks, 2x loop unrolling,
dst-sequential";
void rotate_b32_u2_c(int dim, pixel *src, pixel *dst)
{
    rotate_unified(dim, src, dst, 32, 2, 1);
}

/*********************************************************************
 * register_rotate_functions - Register all of your different versions
 *     of the rotate kernel with the driver by calling the
 *     add_rotate_function() for each test function. When you run the
 *     driver program, it will test and report the performance of each
 *     registered test function.
 *********************************************************************/

void register_rotate_functions()
{
    /* ... Register additional test functions here */
    add_rotate_function(&naive_rotate, naive_rotate_descr);
    add_rotate_function(&rotate_c, rotate_c_descr);

    add_rotate_function(&rotate_b4, rotate_b4_descr);
    add_rotate_function(&rotate_b4_c, rotate_b4_c_descr);
    add_rotate_function(&rotate_b4_u2, rotate_b4_u2_descr);
```

```
    add_rotate_function(&rotate_b4_u2_c, rotate_b4_u2_c_descr);
    add_rotate_function(&rotate_b4_u4, rotate_b4_u4_descr);
    add_rotate_function(&rotate_b4_u4_c, rotate_b4_u4_c_descr);

    add_rotate_function(&rotate_b8, rotate_b8_descr);
    add_rotate_function(&rotate_b8_c, rotate_b8_c_descr);
    add_rotate_function(&rotate_b8_u2, rotate_b8_u2_descr);
    add_rotate_function(&rotate_b8_u2_c, rotate_b8_u2_c_descr);
    add_rotate_function(&rotate_b8_u4, rotate_b8_u4_descr);
    add_rotate_function(&rotate_b8_u4_c, rotate_b8_u4_c_descr);

    add_rotate_function(&rotate_b16, rotate_b16_descr);
    add_rotate_function(&rotate_b16_c, rotate_b16_c_descr);
    add_rotate_function(&rotate_b16_u2, rotate_b16_u2_descr);
    add_rotate_function(&rotate_b16_u2_c, rotate_b16_u2_c_descr);
    add_rotate_function(&rotate_b16_u4, rotate_b16_u4_descr);
    add_rotate_function(&rotate_b16_u4_c, rotate_b16_u4_c_descr);

    add_rotate_function(&rotate_b32, rotate_b32_descr);
    add_rotate_function(&rotate_b32_c, rotate_b32_c_descr);
    add_rotate_function(&rotate_b32_u2, rotate_b32_u2_descr);
    add_rotate_function(&rotate_b32_u2_c, rotate_b32_u2_c_descr);
    add_rotate_function(&rotate_b32_u4, rotate_b32_u4_descr);
    add_rotate_function(&rotate_b32_u4_c, rotate_b32_u4_c_descr);
}

/***************
 * SMOOTH KERNEL
 **************/

/*****************************************************************
 * Various typedefs and helper functions for the smooth function
 * You may modify these any way you like.
 *****************************************************************/

/* A struct used to compute averaged pixel value */
typedef struct {
    int red;
    int green;
    int blue;
    int num;
} pixel_sum;

/* Compute min and max of two integers, respectively */
static int min(int a, int b) { return (a < b ? a : b); }
static int max(int a, int b) { return (a > b ? a : b); }

/*
 * initialize_pixel_sum - Initializes all fields of sum to 0
 */
static void initialize_pixel_sum(pixel_sum *sum)
{
    sum->red = sum->green = sum->blue = 0;
    sum->num = 0;
    return;
}

/*
 * accumulate_sum - Accumulates field values of p in corresponding
 * fields of sum
 */
static void accumulate_sum(pixel_sum *sum, pixel p)
{
```

```
    sum->red += (int) p.red;
    sum->green += (int) p.green;
    sum->blue += (int) p.blue;
    sum->num++;
    return;
}

/*
 * assign_sum_to_pixel - Computes averaged pixel value in current_pixel
 */
static void assign_sum_to_pixel(pixel *current_pixel, pixel_sum sum)
{
    current_pixel->red = (unsigned short) (sum.red/sum.num);
    current_pixel->green = (unsigned short) (sum.green/sum.num);
    current_pixel->blue = (unsigned short) (sum.blue/sum.num);
    return;
}

/*
 * avg - Returns averaged pixel value at (i,j)
 */
static pixel avg(int dim, int i, int j, pixel *src)
{
    int ii, jj;
    pixel_sum sum;
    pixel current_pixel;

    initialize_pixel_sum(&sum);
    for(ii = max(i-1, 0); ii <= min(i+1, dim-1); ii++)
    for(jj = max(j-1, 0); jj <= min(j+1, dim-1); jj++)
        accumulate_sum(&sum, src[RIDX(ii, jj, dim)]);

    assign_sum_to_pixel(&current_pixel, sum);
    return current_pixel;
}

/******************************************************
 * Your different versions of the smooth kernel go here
 ******************************************************/

/*
 * naive_smooth - The naive baseline version of smooth
 */
char naive_smooth_descr[] = "naive_smooth: Naive baseline implementation";
void naive_smooth(int dim, pixel *src, pixel *dst)
{
    int i, j;

    for (i = 0; i < dim; i++)
    for (j = 0; j < dim; j++)
        dst[RIDX(i, j, dim)] = avg(dim, i, j, src);
}



/*
 * refined_smooth_v1 - Copying the whole matrix sequentially for every pixel in kernel
 */
char refined_smooth_v1_descr[] = "Copying the whole matrix sequentially for every pixel in kernel";

typedef struct Coor_struct {
    int r, c;
} Coor;
```

```
Coor kernel_shifts[9] = {
    {-1, -1}, {-1, 0}, {-1, 1},
    {0, -1},  {0, 0},  {0, 1},
    {1, -1},  {1, 0},  {1, 1}
};

pixel_sum src_sum[360000];

void refined_smooth_v1(int dim, pixel *src, pixel *dst) {
    // Clear the source array using memset
    // memset(dst, 0, dim * dim * sizeof(pixel));

    memset(src_sum, 0, dim * dim * sizeof(pixel_sum));

    for(int shift = 0; shift < 9; shift++){
        const int r = kernel_shifts[shift].r, c = kernel_shifts[shift].c;
        const int dst_r_start = (r == -1 ? 1 : 0), dst_r_end = (r == 1 ? dim - 2 : dim - 1);
        const int dst_c_start = (c == -1 ? 1 : 0), dst_c_end = (c == 1 ? dim - 2 : dim - 1);
        for(int i = dst_r_start; i <= dst_r_end; i++){
            for(int j = dst_c_start; j <= dst_c_end; j++){
                accumulate_sum(&src_sum[RIDX(i, j, dim)], src[RIDX(i + r, j + c, dim)]);
            }
        }
    }

    // Convert sum to average by /9
    for (int i = 0; i < dim; i++) {
        for (int j = 0; j < dim; j++) {
            dst[RIDX(i, j, dim)].red = (unsigned short) (src_sum[RIDX(i, j, dim)].red /
src_sum[RIDX(i, j, dim)].num);
            dst[RIDX(i, j, dim)].green = (unsigned short) (src_sum[RIDX(i, j, dim)].green /
src_sum[RIDX(i, j, dim)].num);
            dst[RIDX(i, j, dim)].blue = (unsigned short) (src_sum[RIDX(i, j, dim)].blue /
src_sum[RIDX(i, j, dim)].num);
        }
    }

}

/*
 * refined_smooth_v2 - Added three row buffers and sliding window calculation to prevent excessive
arithmetic and memory access
 */
char refined_smooth_v2_descr[] = "Added three row buffers and sliding window calculation to prevent
excessive arithmetic and memory access";

pixel_sum row_buffer[1600];

#define BIDX(slot, i) ((i) * 3 + (slot)) // slots can be 0,1,2

void write_into_sum(pixel_sum *dst, pixel_sum *src)
{
    dst->red = src->red;
    dst->green = src->green;
    dst->blue = src->blue;
    dst->num = src->num;
}

void remove_sum(pixel_sum *sum, pixel p)
{
    sum->red -= (int) p.red;
    sum->green -= (int) p.green;
    sum->blue -= (int) p.blue;
```

```
    sum->num--;
}

void accumulate_sum2(pixel_sum *dst_sum, pixel_sum sum){
    dst_sum->red += sum.red;
    dst_sum->green += sum.green;
    dst_sum->blue += sum.blue;
    dst_sum->num += sum.num;
}

void calc_row(pixel *src, int i, int dim, int slot) {
    pixel_sum sum;
    initialize_pixel_sum(&sum);

    // Calculating first pixel (j=0)
    accumulate_sum(&sum, src[RIDX(i, 0, dim)]);
    accumulate_sum(&sum, src[RIDX(i, 1, dim)]);
    write_into_sum(&row_buffer[BIDX(slot, 0)], &sum);

    // Calculating second pixel (j=1)
    accumulate_sum(&sum, src[RIDX(i, 2, dim)]);
    write_into_sum(&row_buffer[BIDX(slot, 1)], &sum);

    // Calculating middle pixels (j=1 to dim-2)
    for(int j = 2; j <= dim - 2; j++){
        remove_sum(&sum, src[RIDX(i, j-2, dim)]);
        accumulate_sum(&sum, src[RIDX(i, j+1, dim)]);
        write_into_sum(&row_buffer[BIDX(slot, j)], &sum);
    }

    // Calculating last pixel (j=dim-1)
    remove_sum(&sum, src[RIDX(i, dim-3, dim)]);
    write_into_sum(&row_buffer[BIDX(slot, dim-1)], &sum);
}

void avg_buffers(pixel *dst, int i, int dim, int abandon_slot){
    pixel_sum sum;

    for(int j = 0; j < dim; j++){
        // Clear sum
        initialize_pixel_sum(&sum);

        // Accumulate from the three row buffers
        for(int slot = 0; slot < 3; slot++){
            if(slot == abandon_slot) continue;
            accumulate_sum2(&sum, row_buffer[BIDX(slot, j)]);
        }

        // Assign to dst pixel
        dst[RIDX(i, j, dim)].red = (unsigned short)(sum.red / sum.num);
        dst[RIDX(i, j, dim)].green = (unsigned short)(sum.green / sum.num);
        dst[RIDX(i, j, dim)].blue = (unsigned short)(sum.blue / sum.num);
    }
}

void refined_smooth_v2(int dim, pixel *src, pixel *dst) {
    calc_row(src, 0, dim, 0);
    calc_row(src, 1, dim, 1);
    avg_buffers(dst, 0, dim, 2); // abandon slot 2

    for(int i = 1; i < dim - 1; i++){
        calc_row(src, i + 1, dim, (i + 1) % 3);
        avg_buffers(dst, i, dim, -1); // abandon slot (i + 2) % 3
```

```
    }
    avg_buffers(dst, dim - 1, dim, dim % 3);
}

/*
 * refined_smooth_v2_expanded - Expanded function calls for better pipeline optimization
 */

 char refined_smooth_v2_expanded_descr[] = "Expanded function calls for better pipeline
optimization";

void calc_row_expanded(pixel *src, int i, int dim, int slot) {
    pixel_sum sum;
    initialize_pixel_sum(&sum);
    int src_index;
    int buffer_index;

    // Calculating first pixel (j=0)
    // accumulate_sum(&sum, src[RIDX(i, 0, dim)]);
    src_index = RIDX(i, 0, dim);
    sum.red += src[src_index].red;
    sum.green += src[src_index].green;
    sum.blue += src[src_index].blue;
    sum.num++;
    // accumulate_sum(&sum, src[RIDX(i, 1, dim)]);
    src_index++;
    sum.red += src[src_index].red;
    sum.green += src[src_index].green;
    sum.blue += src[src_index].blue;
    sum.num++;
    // write_into_sum(&row_buffer[BIDX(slot, 0)], &sum);
    buffer_index = BIDX(slot, 0);
    row_buffer[buffer_index].red = sum.red;
    row_buffer[buffer_index].green = sum.green;
    row_buffer[buffer_index].blue = sum.blue;
    row_buffer[buffer_index].num = sum.num;

    // Calculating second pixel (j=1)
    // accumulate_sum(&sum, src[RIDX(i, 2, dim)]);
    src_index++;
    sum.red += src[src_index].red;
    sum.green += src[src_index].green;
    sum.blue += src[src_index].blue;
    sum.num++;
    // write_into_sum(&row_buffer[BIDX(slot, 1)], &sum);
    buffer_index += 3;
    row_buffer[buffer_index].red = sum.red;
    row_buffer[buffer_index].green = sum.green;
    row_buffer[buffer_index].blue = sum.blue;
    row_buffer[buffer_index].num = sum.num;

    int rem_src_index = RIDX(i, -1, dim);
    // Calculating middle pixels (j=1 to dim-2)
    for(int j = 2; j <= dim - 2; j++){
        // remove_sum(&sum, src[RIDX(i, j-2, dim)]);
        rem_src_index++;
        sum.red -= src[rem_src_index].red;
        sum.green -= src[rem_src_index].green;
        sum.blue -= src[rem_src_index].blue;
        sum.num--;
        // accumulate_sum(&sum, src[RIDX(i, j+1, dim)]);
        src_index++;
        sum.red += src[src_index].red;
```

```
        sum.green += src[src_index].green;
        sum.blue += src[src_index].blue;
        sum.num++;
        // write_into_sum(&row_buffer[BIDX(slot, j)], &sum);
        buffer_index += 3;
        row_buffer[buffer_index].red = sum.red;
        row_buffer[buffer_index].green = sum.green;
        row_buffer[buffer_index].blue = sum.blue;
        row_buffer[buffer_index].num = sum.num;
    }

    // Calculating last pixel (j=dim-1)
    // remove_sum(&sum, src[RIDX(i, dim-3, dim)]);
    rem_src_index++;
    sum.red -= src[rem_src_index].red;
    sum.green -= src[rem_src_index].green;
    sum.blue -= src[rem_src_index].blue;
    sum.num--;
    // write_into_sum(&row_buffer[BIDX(slot, dim-1)], &sum);
    buffer_index += 3;
    row_buffer[buffer_index].red = sum.red;
    row_buffer[buffer_index].green = sum.green;
    row_buffer[buffer_index].blue = sum.blue;
    row_buffer[buffer_index].num = sum.num;
}

void avg_2_buffers_expanded(pixel *dst, int i, int dim, int abandon_slot){
    pixel_sum sum;
    int buffer_index;

    for(int j = 0; j < dim; j++){
        // Clear sum
        initialize_pixel_sum(&sum);

        // Accumulate from the three row buffers
        for(int slot = 0; slot < 3; slot++){
            if(slot == abandon_slot) continue;

            buffer_index = BIDX(slot, j);
            sum.red += row_buffer[buffer_index].red;
            sum.green += row_buffer[buffer_index].green;
            sum.blue += row_buffer[buffer_index].blue;
            sum.num += row_buffer[buffer_index].num;
        }

        // Assign to dst pixel
        dst[RIDX(i, j, dim)].red = (unsigned short)(sum.red / sum.num);
        dst[RIDX(i, j, dim)].green = (unsigned short)(sum.green / sum.num);
        dst[RIDX(i, j, dim)].blue = (unsigned short)(sum.blue / sum.num);
    }
}

void avg_3_buffers_expanded(pixel *dst, int i, int dim){
    pixel_sum sum;
    int buffer_index = 0;

    for(int j = 0; j < dim; j++){
        // Clear sum
        initialize_pixel_sum(&sum);

        // Accumulate from the three row buffers
        sum.red += row_buffer[buffer_index].red;
        sum.green += row_buffer[buffer_index].green;
```

```
        sum.blue += row_buffer[buffer_index].blue;
        sum.num += row_buffer[buffer_index].num;

        buffer_index++;
        sum.red += row_buffer[buffer_index].red;
        sum.green += row_buffer[buffer_index].green;
        sum.blue += row_buffer[buffer_index].blue;
        sum.num += row_buffer[buffer_index].num;

        buffer_index++;
        sum.red += row_buffer[buffer_index].red;
        sum.green += row_buffer[buffer_index].green;
        sum.blue += row_buffer[buffer_index].blue;
        sum.num += row_buffer[buffer_index].num;

        // Assign to dst pixel
        dst[RIDX(i, j, dim)].red = (unsigned short)(sum.red / sum.num);
        dst[RIDX(i, j, dim)].green = (unsigned short)(sum.green / sum.num);
        dst[RIDX(i, j, dim)].blue = (unsigned short)(sum.blue / sum.num);

        buffer_index++;
    }
}

void refined_smooth_v2_expanded(int dim, pixel *src, pixel *dst) {
    calc_row_expanded(src, 0, dim, 0);
    calc_row_expanded(src, 1, dim, 1);
    avg_2_buffers_expanded(dst, 0, dim, 2); // abandon slot 2

    for(int i = 1; i < dim - 1; i++){
        calc_row_expanded(src, i + 1, dim, (i + 1) % 3);
        avg_3_buffers_expanded(dst, i, dim); // abandon slot (i + 2) % 3
    }
    avg_2_buffers_expanded(dst, dim - 1, dim, dim % 3);
}

/*
 * smooth - Your current working version of smooth.
 * IMPORTANT: This is the version you will be graded on
 */
char smooth_descr[] = "smooth: Current working version";
void smooth(int dim, pixel *src, pixel *dst)
{

}


/*********************************************************************
 * register_smooth_functions - Register all of your different versions
 *     of the smooth kernel with the driver by calling the
 *     add_smooth_function() for each test function.  When you run the
 *     driver program, it will test and report the performance of each
 *     registered test function.
 *********************************************************************/

void register_smooth_functions() {
    /* ... Register additional test functions here */
    add_smooth_function(&naive_smooth, naive_smooth_descr);
    add_smooth_function(&refined_smooth_v1, refined_smooth_v1_descr);
    add_smooth_function(&refined_smooth_v2, refined_smooth_v2_descr);
    add_smooth_function(&refined_smooth_v2_expanded, refined_smooth_v2_expanded_descr);
}
```